Assignment-9.5

Name:U.Sushmitha

Batch:24

Ht.NO:2303A51705

Problem 1: String Utilities Function

Consider the following Python function: def

reverse_string(text):

return text[::-1]

Task:

1. Write documentation in:

o (a) Docstring o (b) Inline

comments o (c) Google-style

documentation

2. Compare the three documentation styles.

3. Recommend the most suitable style for a utility-based string

library.

```
reverse_string.py > ...
#Docstring
def reverse_string(text):
    """
    This function takes a string as input and returns the reverse of that string.
    """
    return text[::-1]
print(reverse_string.__doc__)

#google style documentation
def reverse_string(text):
    """
    Reverses the given string.
    Args:
        text (str): Input string to be reversed.
    Returns:
        str: Reversed string.
    """
    return text[::-1]
print(reverse_string.__doc__)

#Inline comments
def reverse_string(text):
    # Use slicing with step -1 to reverse the string
    return text[::-1]
print(reverse_string.__doc__)  # This will print None since there is no docstring defined for the function
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Use
op/AI ASSISTANT CODING/reverse_string.py"
This function takes a string as input and returns the reverse of that string.
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY REDDY/OneDrive/
op/AI ASSISTANT CODING/reverse_string.py"
None
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/SONY
op/AI ASSISTANT CODING/reverse_string.py"


Reverses the given string.
Args:
    text (str): Input string to be reversed.
Returns:
    str: Reversed string.
```

Problem 2: Password Strength Checker

Consider the function: def

check_strength(password):

return len(password) >= 8 Task:

1. Document the function using docstring, inline comments, and

   Google style.

2. Compare documentation styles for security-related code.

3. Recommend the most appropriate style.

check_strength.py > ...

```python
#Docstring
def check_strength(password):
    """
    Checks if password length is at least 8 characters.
    """
    return len(password) >= 8
print(check_strength.__doc__)


#Inline comments
def check_strength(password):
    # Password must be at least 8 characters long
    return len(password) >= 8
print(check_strength.__doc__)  # This will print None since there is no docstring defined for the function


#Google style documentation
def check_strength(password):
    """
    Checks whether the password satisfies minimum length requirement.
    Args:
        password (str): Password string.
    Returns:
        bool: True if password length >= 8, else False.
    """
    return len(password) >= 8
print(check_strength.__doc__)
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/pyth
op/AI ASSISTANT CODING/check_strength.py"

Checks if password length is at least 8 characters.
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/pyth
op/AI ASSISTANT CODING/check_strength.py"
None
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Python313/pyth
op/AI ASSISTANT CODING/check_strength.py"

op/AI ASSISTANT CODING/check_strength.py"


Checks whether the password satisfies minimum length requirement.
Args:
    password (str): Password string.
Returns:
    bool: True if password length >= 8, else False.
```

Problem 3: Math Utilities Module Task:

1. Create a module math_utils.py with functions:

o square(n)

o cube(n) o

factorial(n)

2. Generate docstrings automatically using AI tools.

3. Export documentation as an HTML file.

math_utils.py > ...

```python
def square(n):
    """
    Returns the square of a number.
    demonstrates how to use docstrings in Python.
    Parameters:
    n (int): The number to be squared.
    Returns:int: The square of n.
    """
    return n * n

def cube(n):
    """
    Returns the cube of a number.
    demonstrates how to use docstrings in Python.
    Parameters:
    n (int): The number to be cubed.
    Returns:int: The cube of n.
    """
    return n * n * n

def factorial(n):
    """
    Returns the factorial of a number.
    demonstrates how to use docstrings in Python.
    Parameters:
    n (int): The number to calculate the factorial of.
    Returns:int: The factorial of n.
    """
    if n == 0:  # Check if n is 0 and return 1 if it is because factorial of 0 is 1
        return 1  # Factorial of 0 is defined to be 1
    else:
        return n * factorial(n - 1)  # Recursive call to calculate factorial of n
print(square.__doc__)
print(cube.__doc__)
print(factorial.__doc__)
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> & "C:/Users/SONY REDDY/AppData/Local/Programs/Python/Pyt
op/AI ASSISTANT CODING/math_utils.py"

Returns the square of a number.
demonstrates how to use docstrings in Python.
Parameters:
n (int): The number to be squared.
Returns:int: The square of n.


Returns the cube of a number.
demonstrates how to use docstrings in Python.
Parameters:
n (int): The number to be cubed.
Returns:int: The cube of n.


Returns the factorial of a number.
demonstrates how to use docstrings in Python.
Parameters:
n (int): The number to calculate the factorial of.
Returns:int: The factorial of n.
```

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:

o mark_present(student)

o mark_absent(student) o

get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

attendance.py > mark_absent

```python
attendance = {}
def mark_present(student):
    """
    Marks a student as present in the attendance record.
    Parameters:
    student (str): The name of the student to be marked as present.
    """
    attendance[student] = "Present"

def mark_absent(student):
    """
    Marks a student as absent in the attendance record.
    Parameters:
    student (str): The name of the student to be marked as absent.
    """
    attendance[student] = "Absent"

def get_attendance(student):
    """
    Returns the attendance status of a student.
    Parameters:
    student (str): The name of the student whose attendance is to be retrieved.
    Returns:
    str: The attendance status of the student.
    """
    return attendance.get(student, "Not found")
import attendance
help(attendance)
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> python -m pydoc -w attendance
Help on module attendance:

NAME
    attendance

FUNCTIONS
    get_attendance(student)
        Returns the attendance status of a student.
        Parameters:
        student (str): The name of the student whose attendance is to be retrieved.
        Returns:
        str: The attendance status of the student.

    mark_absent(student)
        Marks a student as absent in the attendance record.
        Parameters:
-- More  --
```

Functions

get_attendance(student)
```
Returns the attendance status of a student.
  Parameters:
  student (str): The name of the student whose attendance is to be retrieved.
  Returns:
  str: The attendance status of the student.
```

mark_absent(student)
```
Marks a student as absent in the attendance record.
  Parameters:
  student (str): The name of the student to be marked as absent.
```

mark_present(student)
```
Marks a student as present in the attendance record.
Parameters:
student (str): The name of the student to be marked as present.
```

Data

attendance = {}

Problem 5: File Handling Function

Consider the function: def

read_file(filename): with

open(filename, 'r') as f:

return f.read() Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

read_file.py > ...

```python
# DocString style:
def read_file(course):
    """
    Reads the content of a file and returns it as a string.
    Parameters:
    filename (str): The name of the file to be read.
    Returns:
    str: The content of the file.
    Raises:
    FileNotFoundError: If the specified file does not exist.
    IOError: If an I/O error occurs while reading the file.
    """

    try:
        with open(course, 'r') as f:
            return f.read()
    except FileNotFoundError:
        print(f"Error: The file '{course}' was not found.")
        raise
    except IOError as e:
        print(f"An I/O error occurred: {e}")
        raise

#google style:
def read_file(filename):
    """
    Reads the content of a file and returns it as a string.

    Args:
        filename (str): The name of the file to be read.

    Returns:
        str: The content of the file.

    Raises:
        FileNotFoundError: If the specified file does not exist.
        IOError: If an I/O error occurs while reading the file.
```

```python
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
        raise
    except IOError as e:
        print(f"An I/O error occurred: {e}")
        raise


#Inline comments
def read_file(filename):
    # Open the file in read mode
    # If the file does not exist, Python raises FileNotFoundError
    with open(filename, 'r') as f:
        # Read the entire contents of the file
        return f.read()
```

```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> python -m pydoc -w course
{'name': 'Introduction to Computer Science', 'credits': 4}
Course with ID HIST101 not found in the catalog.

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

None

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

wrote course.html
```


```
PS C:\Users\SONY REDDY\OneDrive\Desktop\AI ASSISTANT CODING> python -m pydoc -w course
{'name': 'Introduction to Computer Science', 'credits': 4}
Course with ID HIST101 not found in the catalog.

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

None

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

wrote course.html
```